

V1: manually extracted from paper

Memory:

Store and maintain information internally for multi-step interaction with the world

- Working memory (active and readily available information as symbolic variables):
 - Perceptual inputs
 - Intermediate reasoning
 - Interact with long-term memories and grounding interface
- Episodic memory:
 - Store experience from earlier decision cycles:
 - Training input-output pairs
 - History event flows
 - Game trajectories from previous episodes
 - Write new experiences from working to episodic memory as a form of learning
- Semantic memory:

- Stores an agent's knowledge about the world and itself.
- Write new knowledge obtained from LLM reasoning into semantic memory as a form of learning

Grounding actions:

Execute external actions and process environmental feedback into working memory as text.

- Physical environments
 - Process perceptual inputs into textual observations
 - Affecting the physical environments via robotic planners that take language-based commands.
 - VLMs are typically used to convert images to text, providing additional context for the LLM
- Dialogue with humans or other agents:
 - Accept instructions or learn from people
 - Ask for help or clarification
 - Entertain or emotionally help people
 - Investigate interaction among multiple language agents for social simulation
 - Multi-agent debate
 - Multi-agent collaborative task solving
- Digital environment:
 - Interacting with games, APIs, and websites as well as general code execution
 - Cheaper and faster than physical or human interaction
 - Augmentation of external knowledge or computation, stateless digital APIs

Retrieval actions:

- Loads code-based skills from a skill library via dense retrieval to interact with the Minecraft world
- Retrieves relevant events from episodic memory via a combination of recency, importance and relevance scores
- Leverage library documents to assist code generation
- Retrieving knowledge from semantic memory

Reasoning actions:

Process the contents of working memory to generate new information.

- Reads from and writes to working memory
- Summarize and distill insights about the most recent observation, the most recent trajectory, or information retrieved from long-term memory

- Using the results as additional context for subsequent LLM calls

Learning actions:

- Updating episodic memory with experience
 - Store episodic trajectories to update a parametric policy
 - Update a parametric policy
 - Establish a non-parametric policy
 - Added experiences in episodic memory may be retrieved later as examples and bases for reasoning or decision-making
- Updating semantic memory with knowledge
 - Reason about raw experiences and store the resulting inferences in semantic memory
 - Reflect on failed episodes and stores the results
 - Use Vision-language models to build a semantic map of the environment
- Updating LLM parameters.
 - Fine-tuning during agent's lifetime. Accomplished via supervised or imitation learning, reinforcement learning from environment feedback, Human feedback,
 - LLM self-improvement methods, consistency to select generations to fine-tune on.
 - Extended to use environmental feedback instead. Fine-tunes a small language model on high-scoring trajectories stored in episodic memory
 - Robust exploitation policy to reach exploration frontiers
 - Utilize smaller subtask-specific LLMs
 - Allow language agents to autonomously determine when and how to fine-tune their LLMs.
- Updating agent code (procedural memory). Update their source code, thus modifying the implementation of various procedures
 - Updating reasoning. Infers prompt instructions from input-output examples, then uses these instructions as part of the llm prompt to assist task solving.
 - Prompt update can be seen as a form of learning to reason.
 - Updating grounding.
 - Code-based skills.
 - Maintains a curriculum library.

- Updating retrieval. Retrieval is usually considered a basic action designed with some fixed implementation. Learn better procedures
- Updating learning or decision-making.
 - Learn new procedures for learning or decision-making

Decision-making

How should a language agent choose which action to apply?

Plan:

- Proposal: one or more action candidates
 - Evaluate
 - Select actions
 - generate one or more action candidates
- Evaluate: assigns a value to each
 - Heuristic rules
 - LLM values
 - Learned values
 - LLM reasoning
 - Help evaluate actions by internally simulating their grounding feedback from the external world
- Select:
 - Selects one to execute or rejects them
 - Selection may occur via argmax, softmax, or an alternative such as majority vote

Execution:

- Depending on the agent implementation
- External grounding action:
- API call
- Internal learning action